

CS532 HW3

Ethan Reinhart

November 2025

Problem 4

We have three input ports and three output ports (X, Y, Z). The packets in the input queues are:

Input 1: X Input 2: X, Y Input 3: Z, Y .

Thus the total number of packets for each output is

$$X : 2, \quad Y : 2, \quad Z : 1.$$

Each time slot the crossbar can send at most one packet to each output, but all three outputs can be used simultaneously.

Minimum number of slots

A lower bound is obtained by dividing total packets by the number of outputs:

$$\left\lceil \frac{5}{3} \right\rceil = 2 \text{ slots.}$$

We can achieve 2 slots, e.g.

- Slot 1: Input 1 $\rightarrow X$, Input 2 $\rightarrow Y$, Input 3 $\rightarrow Z$.
- Slot 2: Input 2 $\rightarrow X$, Input 3 $\rightarrow Y$.

So the **minimum** number of slots is

$$\boxed{2}.$$

Maximum number of slots (worst-case scheduling)

We want to create as much output contention as possible without ever leaving a non-empty input idle. For example, order the queues as

Input 1: X ; Input 2: X, Y ; Input 3: Y, Z .

One possible evolution:

- Slot 1: send X from input 1 and Y from input 3. Remaining: X, Y, Z .

- Slot 2: send X from input 2 and Z from input 3. Remaining: Y .
- Slot 3: send Y from input 2.

We cannot do worse than 3 slots without idling an input that has a packet.
Thus the **maximum** number of slots (under the constraints) is

$$\boxed{3}.$$

Problem 6

From the figure, arrival times (in time units) are:

$$\begin{aligned} t = 0 : & \quad 1, 2 \\ t = 1 : & \quad 3, 4 \\ t = 2 : & \quad 6 \\ t = 3 : & \quad 5, 7 \\ t = 5 : & \quad 8, 9 \\ t = 7 : & \quad 10 \\ t = 8 : & \quad 11, 12. \end{aligned}$$

Each packet requires exactly one time unit of service. Packet 1 starts service at $t = 0$ and departs at $t = 1$.

For each packet i we will use:

$$A_i = \text{arrival time}, \quad S_i = \text{service start time}, \quad D_i = \text{departure time}, \quad W_i = S_i - A_i, \quad T_i = D_i - A_i.$$

Note that $T_i = W_i + 1$ since service time is 1.

(a) FIFO

First-come first-served. The resulting times are:

Pkt	A_i	S_i	D_i	W_i	T_i
1	0	0	1	0	1
2	0	1	2	1	2
3	1	2	3	1	2
4	1	3	4	2	3
5	3	5	6	2	3
6	2	4	5	2	3
7	3	6	7	3	4
8	5	7	8	2	3
9	5	8	9	3	4
10	7	9	10	2	3
11	8	10	11	2	3
12	8	11	12	3	4

The delay requested is W_i , the time from arrival until the beginning of transmission. The average is

$$\frac{\sum_{i=1}^{12} W_i}{12} = \frac{23}{12} \approx 1.92.$$

(b) Priority (odd = high, even = low)

Odd-numbered packets (1,3,5,7,9,11) have strict priority over even-numbered packets (2,4,6,8,10,12). Service is non-preemptive: once a packet starts, we finish it.

The schedule:

Pkt	A_i	S_i	D_i	W_i	T_i
1	0	0	1	0	1
2	0	2	3	2	3
3	1	1	2	0	1
4	1	6	7	5	6
5	3	3	4	0	1
6	2	7	8	5	6
7	3	4	5	1	2
8	5	9	10	4	5
9	5	5	6	0	1
10	7	10	11	3	4
11	8	8	9	0	1
12	8	11	12	3	4

Again, the required delay is W_i .

$$W_{\text{avg}} = \frac{23}{12} \approx 1.92, \quad T_{\text{avg}} = \frac{35}{12} \approx 2.92.$$

(c) Round Robin between classes

Class 1: packets 1,2,3,6,11,12. Class 2: packets 4,5,7,8,9,10.

Each class has its own FIFO queue. The scheduler alternates between non-empty class 1 and class 2 queues (C1, C2, C1, C2, ...).

This yields departure order

$$1, 4, 2, 5, 3, 7, 6, 8, 11, 9, 12, 10,$$

and the following times:

Pkt	A_i	S_i	D_i	W_i	T_i
1	0	0	1	0	1
2	0	2	3	2	3
3	1	4	5	3	4
4	1	1	2	0	1
5	3	3	4	0	1
6	2	6	7	4	5
7	3	5	6	2	3
8	5	7	8	2	3
9	5	9	10	4	5
10	7	11	12	4	5
11	8	8	9	0	1
12	8	10	11	2	3

For part (c) the requested delay is the time from arrival to departure, i.e. T_i :

$$T_{\text{avg}} = \frac{35}{12} \approx 2.92.$$

(d) Weighted Fair Queueing

Odd packets (1,3,5,7,9,11) are in class 1 with weight 2. Even packets (2,4,6,8,10,12) are in class 2 with weight 1.

A simple approximation to WFQ is a weighted round-robin pattern C1, C1, C2, C1, C1, C2, ..., skipping empty classes but keeping the cycle.

This gives departure order

1, 3, 2, 5, 7, 4, 9, 6, 8, 11, 10, 12,

and

Pkt	A_i	S_i	D_i	W_i	T_i
1	0	0	1	0	1
2	0	2	3	2	3
3	1	1	2	0	1
4	1	5	6	4	5
5	3	3	4	0	1
6	2	7	8	5	6
7	3	4	5	1	2
8	5	8	9	3	4
9	5	6	7	1	2
10	7	10	11	3	4
11	8	9	10	1	2
12	8	11	12	3	4

$$T_{\text{avg}} = \frac{35}{12} \approx 2.92.$$

(e) Observation

In all four cases (FIFO, strict priority, RR, WFQ) the average delay is the same:

$$W_{\text{avg}} = \frac{23}{12} \approx 1.92, \quad T_{\text{avg}} = \frac{35}{12} \approx 2.92.$$

Scheduling only reallocates which packets wait longer or shorter; but because all packets have equal service time and the server is work-conserving (never idle while packets are queued), the average delay over all packets is unchanged.

Problem 15(a)

We must allocate subnets inside the block

$$214.97.254.0/23$$

which covers addresses 214.97.254.0 through 214.97.255.255 (512 addresses).

Requirements:

- Subnet A: at least 250 interfaces.
- Subnet B: at least 120 interfaces.
- Subnet C: at least 120 interfaces.
- Subnets D, E, F: two interfaces each (router–router links).

A convenient allocation is:

- **Subnet A** (top LAN): use one /24

$$214.97.255.0/24$$

which provides 256 addresses (≥ 250).

- **Subnet C** (bottom-left LAN): use a /25

$$214.97.254.128/25$$

giving 128 addresses (≥ 120).

- **Subnet B** (bottom-right LAN): use the other /25 but carve out the first /29 for the router-only links:

$$214.97.254.0/25 - 214.97.254.0/29$$

The /25 has 128 addresses; removing the first 8 addresses (the /29) leaves 120 usable host addresses.

- Router-only subnets, each with two interfaces:

$$D : 214.97.254.0/31 \quad (\text{addresses } .0, .1)$$

$$E : 214.97.254.2/31 \quad (\text{addresses } .2, .3)$$

$$F : 214.97.254.4/31 \quad (\text{addresses } .4, .5)$$

each /31 supports exactly two interface addresses, suitable for point-to-point links.

All six subnets lie within 214.97.254.0/23 and satisfy the interface requirements.

Problem 20

Both peers, Arnold and Bernard, are behind separate NATs.

To establish a TCP connection, one host must send a SYN to the other's (public IP, port). A NAT, however, only forwards an incoming SYN to an internal host if it already has a mapping in its translation table (typically created by an earlier outbound connection from the internal host or by explicit port forwarding).

Here:

- Arnold cannot connect directly to Bernard's private address, since it is not globally routable.
- Arnold also cannot reliably connect to Bernard's NAT public address, because Bernard's NAT will drop unsolicited inbound SYNs when there is no existing mapping.
- Bernard is in the same situation with respect to Arnold.

One might try to use a rendezvous server: both Arnold and Bernard open TCP connections to a well-known public server, learn each other's external (IP,port), and then attempt "simultaneous open" or TCP hole punching. However, with generic NAT behavior and no control over port mappings or firewall rules, there is no guarantee that these incoming SYNs will be accepted by either NAT.

Therefore, without any application-specific NAT configuration, and with both peers behind typical NATs, Arnold cannot in general establish a direct TCP connection to Bernard. Some remedy is necessary, such as:

- Configuring port forwarding or static NAT rules on at least one NAT.
- Using a publicly reachable relay server (both peers connect outward to it, and it forwards traffic between them).